

SUPERSHELF STORE

Project Documentation

E-Commerce Single-Page Application — React 19 + Vite 7

Compiled Documentation Set

Table of Contents

Section	Description	Page
1. Project Planning & Management	Proposal, Timeline, Roles, Risks & KPIs	3
2. Literature Review	Technical Foundations, Alternatives & Future Work	7
3. Requirements Gathering	Stakeholders, User Stories, Functional & Non-functional Requirements	10
4. System Analysis & Design	Architecture, Data Model, DFDs, UI/UX & Deployment	16
5. Implementation	Code Standards & Best Practices	30
6. Testing & Quality Assurance	Test Cases, Test Plan & Bug Reports	33

SUPERSHELF STORE

Project Planning & Management

Proposal, Timeline, Roles, Risks & KPIs

E-Commerce Single-Page Application — React 19 + Vite 7

Team: Abdallah Okasha, Doha, Mohamed, Kerollos

2026

1. Project Planning & Management

Project Proposal

SuperShelf Store is a single-page e-commerce web application built with React 19 and Vite 7. The application enables users to browse a catalog of products (including books and API-sourced inventory), manage a persistent shopping cart, create accounts, place orders, track purchase history, and maintain a wishlist. An admin dashboard provides tools for managing users, viewing orders, and adding custom products to the catalog. The entire application runs client-side with data persisted through the browser's localStorage and supplemented by the DummyJSON public API for product data.

Key Objectives:

- Deliver a fully functional e-commerce experience without a dedicated backend server
- Provide a responsive, mobile-friendly user interface using Bootstrap 5
- Demonstrate component-based architecture with React Context for state management
- Support role-based access with admin and user tiers
- Enable offline resilience through local fallback data and localStorage persistence

Project Plan

MILESTONE	DELIVERABLE	TIMELINE
Repository & Scaffolding	GitHub repo, Vite + React setup, ESLint config	Week 1
Static HTML/CSS Prototype	Vanilla HTML/CSS/JS proof-of-concept, CSS styling	Week 1–2
React Migration	Migrate to React SPA with component architecture	Week 3
Routing & Navigation	React Router setup, Navbar with profile dropdown	Week 3
Core E-commerce Features	Product listing, product detail, shopping cart	Week 4
User Authentication	Login/Signup with localStorage persistence, role-based access	Week 4
Checkout & Orders	Checkout form with validation, order history, order cancellation	Week 5
Admin Dashboard	User management, order processing, product management	Week 5
Advanced Features	Wishlist, books catalog, hot deals, breadcrumbs	Week 6

MILESTONE	DELIVERABLE	TIMELINE
UI Polish & Responsiveness	Bootstrap integration, responsive breakpoints, styling	Week 6
Admin Enhancements	Manage Items page (add + delete products), password UX improvements	Week 7
Testing & Documentation	Manual testing, bug fixes, complete documentation	Week 7

Task Assignment & Roles

TEAM MEMBER	ROLE	RESPONSIBILITIES
Abdallah Okasha	Lead Developer & Architect	React architecture, Context/state management, admin dashboard (users, orders), checkout flow, order tracking, dummyJSON API integration, GitHub repo management, documentation
Doha	Frontend Developer	Product listing pages, UI components, CSS styling, responsive design, home page hero section
Mohamed	Frontend Developer	Books catalog, hot deals section, navigation, product detail pages
Kerollos	Frontend Developer	Initial HTML/CSS/JS prototype, file organization, style.css, base styling

Risk Assessment & Mitigation Plan

RISK	PROBABILITY	IMPACT	MITIGATION STRATEGY
DummyJSON API downtime	Low	High (product catalog unavailable)	Local fallback products array + offline books catalog from static data
localStorage data loss (clear/corrupt)	Low	Medium (cart, orders, users lost)	Validation on every localStorage read with try/catch and default values
Browser compatibility issues	Low	Medium	Bootstrap 5 provides cross-browser consistency; standard ES modules used
Plaintext password storage	High (by design)	High	Client-side demo only — no real authentication; password strength validation enforced on signup
No backend server for order	N/A	N/A	All order logic runs client-side; orders stored in localStorage with status

RISK	PROBABILITY	IMPACT	MITIGATION STRATEGY
processing			tracking
XSS via product data	Low	Medium	DummyJSON is a controlled API; no <code>dangerouslySetInnerHTML</code> used; text content rendered safely
Accidental dist/commits	Medium	Low	<code>.gitignore</code> includes <code>dist</code> ; team education on commit practices

KPIs

METRIC	TARGET
Production build size (JS)	< 450 KB gzipped
Page load time (production)	< 2 seconds
Product search/filter response	Instant (< 100ms, client-side)
Cart and checkout operations	Instant (local operations)
Responsive breakpoints	4 tiers: ≥1200px, ≥992px, ≥768px, ≥480px
API fallback coverage	100% — app works fully offline with local data
Form validation coverage	100% — login, signup, checkout, add-item all validated
Lighthouse performance score target	≥ 80

SUPERSHELF STORE

Literature Review

Technical Foundations, Alternatives & Future Work

E-Commerce Single-Page Application — React 19 + Vite 7

Team: Abdallah Okasha, Doha, Mohamed, Kerollos

2026

2. Literature Review

Technical Foundations & Related Work

SuperShelf Store draws on established patterns in modern single-page application (SPA) development. The project evaluates and applies several key technologies and architectural decisions:

React 19 with Vite 7: React's component model enables reusable, composable UI elements. Vite provides fast development server startup with Hot Module Replacement (HMR) and optimized production builds using Rollup. The combination yields a modern developer experience with sub-second rebuild times and efficient bundle output.

React Context API vs. External State Libraries: Rather than adopting Redux or Zustand, the project uses React's built-in Context API (AuthContext, CartContext). This is appropriate for the application's scale: two independent state domains (authentication and cart) with simple data flows. Context avoids the boilerplate overhead of Redux while remaining sufficient for this scope.

Client-Side Persistence with localStorage: The project stores users, cart items, orders, wishlists, and custom products in the browser's localStorage. This pattern is well-documented for prototypal and demo applications where no backend is available. The approach mirrors patterns used in Progressive Web Applications (PWAs) for offline data availability, though without Service Worker caching.

Bootstrap 5 for Responsive UI: Bootstrap 5 provides a mature grid system, utility classes, and pre-built components (cards, tables, forms, toasts, badges, modals). The project uses Bootstrap's CSS framework directly rather than a React-specific wrapper like react-bootstrap, favoring utility classes and inline styles for custom overrides.

DummyJSON as a Data Source: The DummyJSON API (dummyjson.com) is a free RESTful fake API commonly used for prototyping. It provides product data, categories, and search across 100+ products. The project falls back to hardcoded local data when the API is unreachable, a pattern recommended for resilient frontend applications.

Client-Side Form Validation: The project implements real-time and submission-time form validation for signup (password strength, email format, confirm match) and checkout (card number formatting, expiry validation, required fields). This follows UX best practices of providing immediate feedback without server round-trips.

Evaluation Against Alternatives

APPROACH	CHOSEN	ALTERNATIVE	RATIONALE
Build tool	Vite 7	Create React App	Faster dev server, better ESM support, smaller bundle
State management	React Context	Redux / Zustand	Simpler API, sufficient for 2 context domains
CSS framework	Bootstrap 5 CSS	Tailwind / Material UI	Familiar grid system, fast prototyping, no wrapper dependency
Data persistence	localStorage	IndexedDB / API backend	Simplicity, no server needed, suitable for demo scope
API source	DummyJSON	Custom backend	Zero server cost, sufficient product data for demonstration
Routing	react-router-dom v7	Manual SPA routing	Industry standard, declarative route definitions

Suggested Improvements for Future Iterations

1. **Backend Integration:** Replace localStorage with a proper backend (Node.js + Express + MongoDB/PostgreSQL) for real authentication (JWT), payment processing (Stripe), and persistent order management.
2. **Automated Testing:** Add unit tests (Vitest/Jest) for utility functions and context providers, integration tests (React Testing Library) for critical flows, and end-to-end tests (Playwright/Cypress) for checkout and admin flows.
3. **Security Hardening:** Implement HTTPS, helmet-style headers, input sanitization, rate limiting, and CSRF protection in a backend version.
4. **Accessibility Audit:** Conduct WCAG 2.1 AA audit; add ARIA labels, keyboard navigation improvements, and screen reader support.
5. **Performance Optimization:** Code-split route-level chunks with `React.lazy`, add image optimization (lazy loading, WebP format), and implement Service Worker caching for full PWA support.
6. **Internationalization:** Add i18n support for the Arabic book catalog's full context, given that many book titles and descriptions are in Arabic.

SUPERSHELF STORE

Requirements Gathering

Stakeholders, User Stories, Functional & Non-functional
Requirements

E-Commerce Single-Page Application — React 19 + Vite 7
Team: Abdallah Okasha, Doha, Mohamed, Kerollos
2026

3. Requirements Gathering

Stakeholder Analysis

STAKEHOLDER	ROLE	NEEDS & EXPECTATIONS
End User (Shopper)	Primary user	Browse products by category, view details, add to cart, checkout securely, track orders, save wishlist items, create/manage account
Admin (Store Owner)	Administrative user	Add custom products to catalog, delete products, manage registered users (promote/revoke), process orders (Pending → Processing → Shipped)
Developer	Technical stakeholder	Clean modular codebase, consistent component patterns, well-documented API integration, version-controlled collaboration
Lecturer/Evaluator	Academic stakeholder	Complete documentation covering all SDLC phases, working demonstration of all features, evidence of team collaboration via Git history

User Stories & Use Cases

ID	USER STORY	PRIORITY	ACCEPTANCE CRITERIA
US-01	As a shopper, I want to browse products grouped by category so I can quickly find relevant items.	High	Category sidebar with 8 categories; clicking a category filters the grid
US-02	As a shopper, I want to filter products by price range so I can shop within my budget.	High	Min/Max price inputs; products filtered in real-time; reset button clears filters
US-03	As a shopper, I want to view full product details (image, price, description) before deciding.	High	Clicking a product card opens a detail page with image, price, description, and Add to Cart
US-04	As a shopper, I want to add products to a shopping cart so I can purchase multiple items together.	High	Add to Cart button on product cards and detail page; toast notification on success
US-05	As a shopper, I want to view my cart, adjust quantities, and remove items.	High	Cart page lists items with quantity, line total, remove button, and total price

ID	USER STORY	PRIORITY	ACCEPTANCE CRITERIA
US-06	As a shopper, I want to complete checkout with shipping info and payment.	High	Checkout form with name, address, phone; card or cash payment; validation on all fields
US-07	As a shopper, I want to create an account so my orders and data persist.	High	Signup form with email, password (strength requirements), confirm password; auto-login on success
US-08	As a shopper, I want to log in to access my orders and wishlist.	High	Login form validates credentials against localStorage users; role-based redirect
US-09	As a shopper, I want to save products to a wishlist so I can purchase them later.	Medium	Heart button on product cards; dedicated Wishlist page with remove functionality
US-10	As a shopper, I want to view my order history with status tracking.	High	Order History page filtered by user email; shows date, items, total, status badge
US-11	As a shopper, I want to cancel a pending order.	Medium	Cancel button on pending orders with confirmation dialog; status changes to Cancelled
US-12	As an admin, I want to add new products to the catalog.	High	Add New form in Manage Items page with title, price, category, image URL, description
US-13	As an admin, I want to delete products from the catalog.	High	Delete button per product in Manage Items table; API products hidden; custom products removed from localStorage
US-14	As an admin, I want to process customer orders (mark as Processing/Shipped).	High	Accordion in Dashboard with order list; Mark Processing / Mark Shipped buttons
US-15	As an admin, I want to manage users (promote to admin, revoke access).	High	Users table with Promote and Revoke actions; confirmation dialogs; hardcoded admin protected
US-16	As a user, I want the site to work on my phone so I can shop anywhere.	Medium	Responsive grid (4→3→2→1 columns); mobile-friendly navbar; touch-friendly button sizes
US-17	As a user, I want to see hot deals on the home page.	Low	Hottest Deals section with discount badges, countdown timer, savings display
US-18	As a user, I want to browse a dedicated books catalog.	Medium	Books tab in sidebar with 20 Arabic/international books; dedicated Books page

Functional Requirements

ID	REQUIREMENT	MODULE	PRIORITY
FR-01	Display product catalog with images, prices, discounts, and pagination	Products	High
FR-02	Filter products by category (8 categories including Books)	Products	High
FR-03	Filter products by price range (min/max inputs)	Products	High
FR-04	Display paginated results with configurable items per page (8/16/24)	Products	High
FR-05	Show product detail with large image, price, description, brand/author	ProductDetails	High
FR-06	Scroll-to-top on page change	Products	Medium
FR-07	Add products to cart with quantity increment for duplicates	CartContext	High
FR-08	Remove individual items from cart	Cart	High
FR-09	Clear entire cart	Cart	Medium
FR-10	Display cart total price	Cart	High
FR-11	Checkout form with shipping fields (name, address, phone)	Checkout	High
FR-12	Payment method selection (Credit/Debit Card or Cash on Delivery)	Checkout	High
FR-13	Credit card input with auto-formatting and brand detection	Checkout	Medium
FR-14	Expiry date validation (month range + future date check)	Checkout	Medium
FR-15	Order creation with status "Pending", saved to localStorage	Checkout	High
FR-16	User registration with email format and password strength validation	Login/Signup	High
FR-17	User login with credential verification against localStorage	Login/Signup	High

ID	REQUIREMENT	MODULE	PRIORITY
FR-18	Live password strength checklist (length, upper, lower, digit)	Login/Signup	Medium
FR-19	Role-based access control (user vs admin)	AuthContext	High
FR-20	Admin dashboard with accordion sections	Dashboard	High
FR-21	Manage Items page with All Items table and Add New form	Manageltems	High
FR-22	Delete API products (client-side hide) and custom products (localStorage)	Manageltems	High
FR-23	Manage users table with Promote and Revoke actions	Dashboard	High
FR-24	Order management with status transitions (Pending → Processing → Shipped)	Dashboard	High
FR-25	User order history filtered by email	OrderHistory	High
FR-26	Cancel pending orders with confirmation	OrderHistory	Medium
FR-27	Persistent wishlist with add/remove toggles	Wishlist / Products	Medium
FR-28	Toast notification system for cart actions	CartMessage	Low
FR-29	Breadcrumb navigation on all pages	Breadcrumbs	Medium
FR-30	Responsive navigation with profile dropdown and click-outside close	Navbar	High
FR-31	Hot deals section on home page with discount badges	Home	Low
FR-32	Hot deals Add to Cart (direct add, no redirect)	Home	Medium
FR-33	About page with store information	About	Low
FR-34	Custom products merged into product grid alongside API and books	Products	High
FR-35	Custom products handled in product detail page	ProductDetails	High

Non-functional Requirements

ID	REQUIREMENT	CRITERIA	MEASUREMENT
NFR-01	Performance	Product page loads and filters render in under 100ms	Browser DevTools performance tab
NFR-02	Responsiveness	Layout adapts at 1200px, 992px, 768px, 480px breakpoints	Manual resize testing
NFR-03	Availability	Application functions fully with local data when API is unreachable	Disconnect network, verify local fallback
NFR-04	Usability	Bootstrap 5 UI with consistent navigation, clear CTAs, form validation feedback	User walkthrough
NFR-05	Maintainability	Components follow single-responsibility pattern; contexts separate concerns	Code review
NFR-06	Compatibility	Works on Chrome, Firefox, Edge latest versions	Cross-browser manual testing
NFR-07	Security (Demo)	Password strength enforced; email format validated; no XSS vectors	Code inspection
NFR-08	Data Persistence	Cart, orders, users, wishlist survive page refresh	localStorage verification
NFR-09	Build Quality	Production build completes without errors; JS bundle under 450 KB	<code>npm run build</code> output
NFR-10	Error Handling	API failures caught and logged; localStorage reads wrapped in try/catch	Code review

SUPERSHELF STORE

System Analysis & Design

Architecture, Data Model, DFDs, UI/UX & Deployment

E-Commerce Single-Page Application — React 19 + Vite 7

Team: Abdallah Okasha, Doha, Mohamed, Kerollos

2026

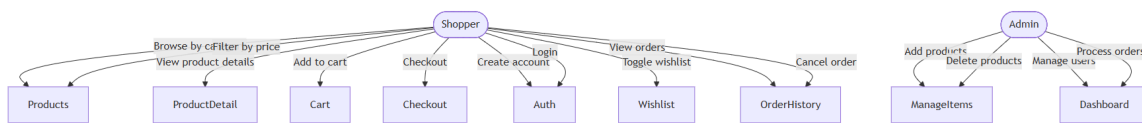
4. System Analysis & Design

4.1 Problem Definition & Architecture

Problem Statement & Objectives

Online shopping has become ubiquitous, yet many demonstration e-commerce projects require complex backend infrastructure. SuperShelf Store aims to prove that a fully functional e-commerce experience — including product browsing, cart management, user authentication, order processing, and administrative controls — can be delivered entirely as a client-side single-page application. The project demonstrates modern React patterns, responsive design, and resilient data handling without a dedicated server.

Use Case Diagram

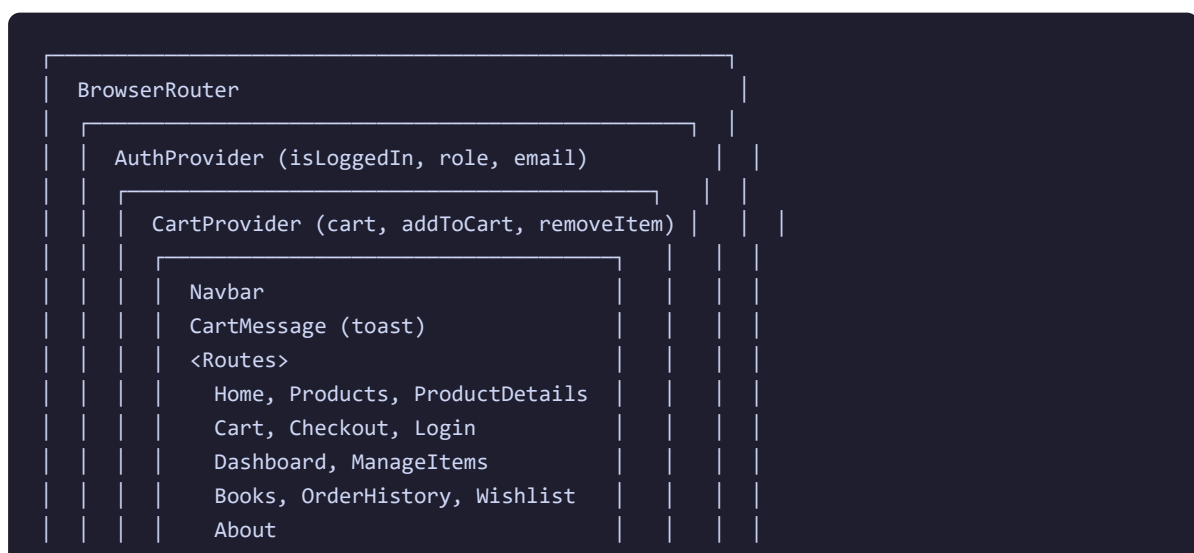


Actors:

- **Shopper:** Browses products, manages cart, checks out, tracks orders, manages wishlist
- **Admin:** All shopper actions plus product management, user management, order processing

Software Architecture

Architecture Pattern: Component-based SPA with Context-driven state management





Data Flow Pattern: Unidirectional — Components dispatch actions to Context providers, which update state and persist to localStorage. Child components re-render via Context subscriptions.

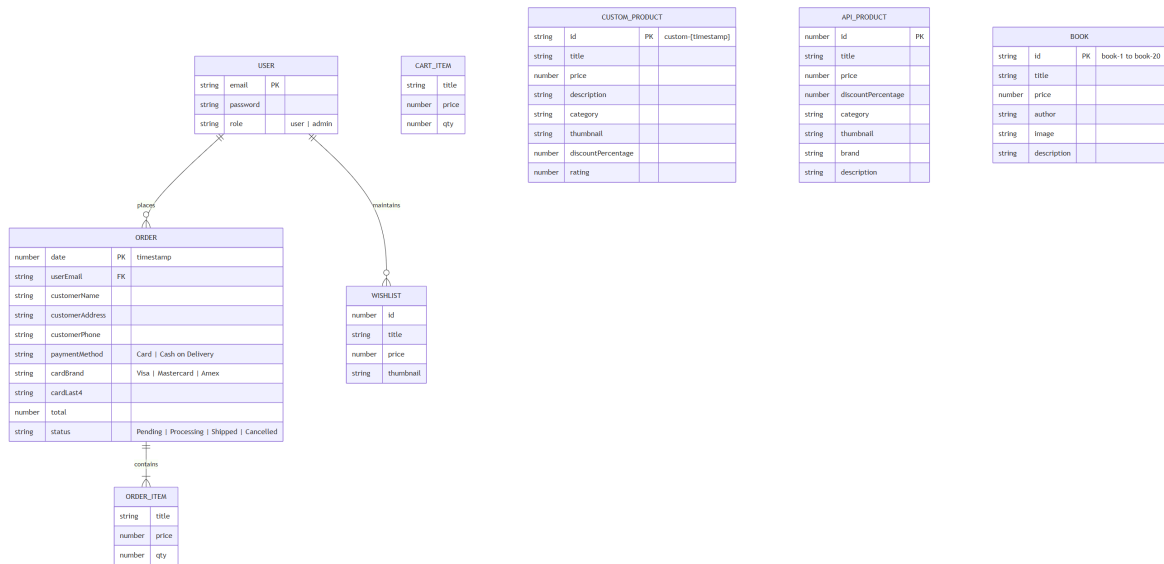
Route Structure:

PATH	COMPONENT	ACCESS
/	Home	Public
/products	Products	Public
/products?category={slug}	Products	Public
/product?id={id}	ProductDetails	Public
/book?id={id}	ProductDetails	Public
/Books	Books	Public
/cart	Cart	Public
/checkout	Checkout	Public
/login	Login	Public
/dashboard	Dashboard	Admin only
/manage-items	Manageltems	Admin only
/orders	OrderHistory	Authenticated users
/wishlist	Wishlist	Authenticated users
/about	About	Public

4.2 Database Design & Data Modeling

The project has no relational database. All persisted data lives in the browser's localStorage or is fetched from the DummyJSON API. Below is the logical data model.

ER Diagram



Logical Schema (localStorage Keys)

KEY	TYPE	DESCRIPTION	EXAMPLE VALUE
users	Array of User objects	Registered accounts	[{ email: "a@b.com", password: "Pass1234", role: "user" }]
orderHistory	Array of Order objects	All placed orders	[{ customer: {...}, payment: {...}, items: [...], total: 49.99, status: "Pending", userEmail: "a@b.com" }]
cart	Array of CartItem objects	Current user's cart	[{ title: "iPhone 9", price: 549, qty: 1 }]
wishlist	Array of WishlistItem objects	Saved items	[{ id: 1, title: "iPhone 9", price: 549, thumbnail: "..." }]
customProducts	Array of CustomProduct objects	Admin-added products	[{ id: "custom-1234", title: "My Product", price: 25, category: "beauty", thumbnail: "..." }]
isLoggedIn	String "true" or "false"	Auth flag	"true"
role	String	Current user role	"admin" or "user"
userEmail	String	Current user email	"a@b.com"

Static Data Files

FILE	DESCRIPTION	RECORDS
<code>src/data/HotDeals.js</code>	4 featured deals for home page	4 products
<code>src/data/booksData.js</code>	20 Arabic/international books	20 books
<code>src/pages/Products.jsx</code> (<code>fallbackProducts</code>)	12 hardcoded products for offline fallback	12 products

Physical Schema Details

User:

```
{ "email": "string (unique)", "password": "string (plaintext)", "role": "'user' | 'admin'" }
```

Order:

```
{
  "customer": { "name": "string", "address": "string", "phone": "string" },
  "payment": { "method": "'Card' | 'Cash on Delivery'", "brand": "string?", "last4": "string?" },
  "date": "number (Date.now())",
  "items": [{ "title": "string", "price": "number", "qty": "number" }],
  "total": "number",
  "status": "'Pending' | 'Processing' | 'Shipped' | 'Cancelled'",
  "userEmail": "string"
}
```

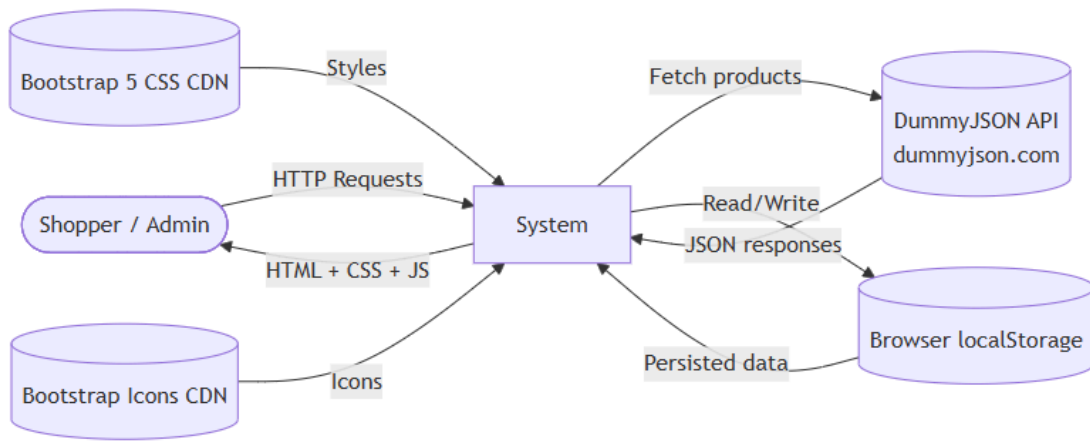
Custom Product:

```
{
  "id": "'custom-' + Date.now()",
  "title": "string",
  "price": "number",
  "description": "string",
  "category": "string",
  "thumbnail": "string (URL)",
  "discountPercentage": 0,
  "rating": 0
}
```

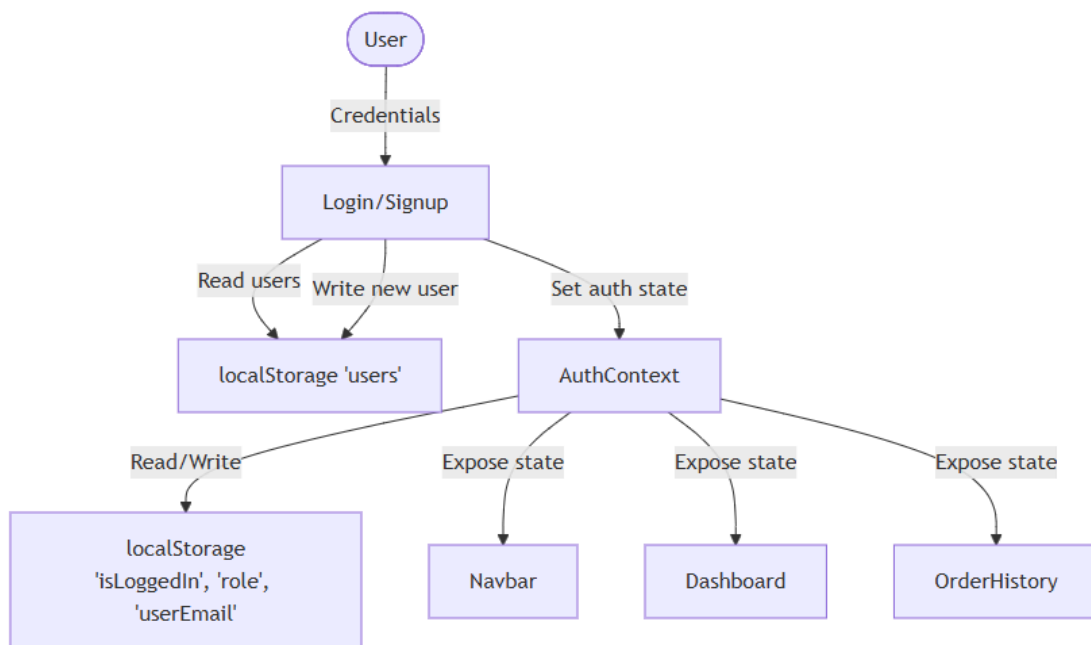
Normalization Note: The data model is denormalized by design (e.g., order items are embedded within orders). This is appropriate for localStorage-based persistence where joins are not possible. In a production backend, orders and order_items would be separate tables.

4.3 Data Flow & System Behavior

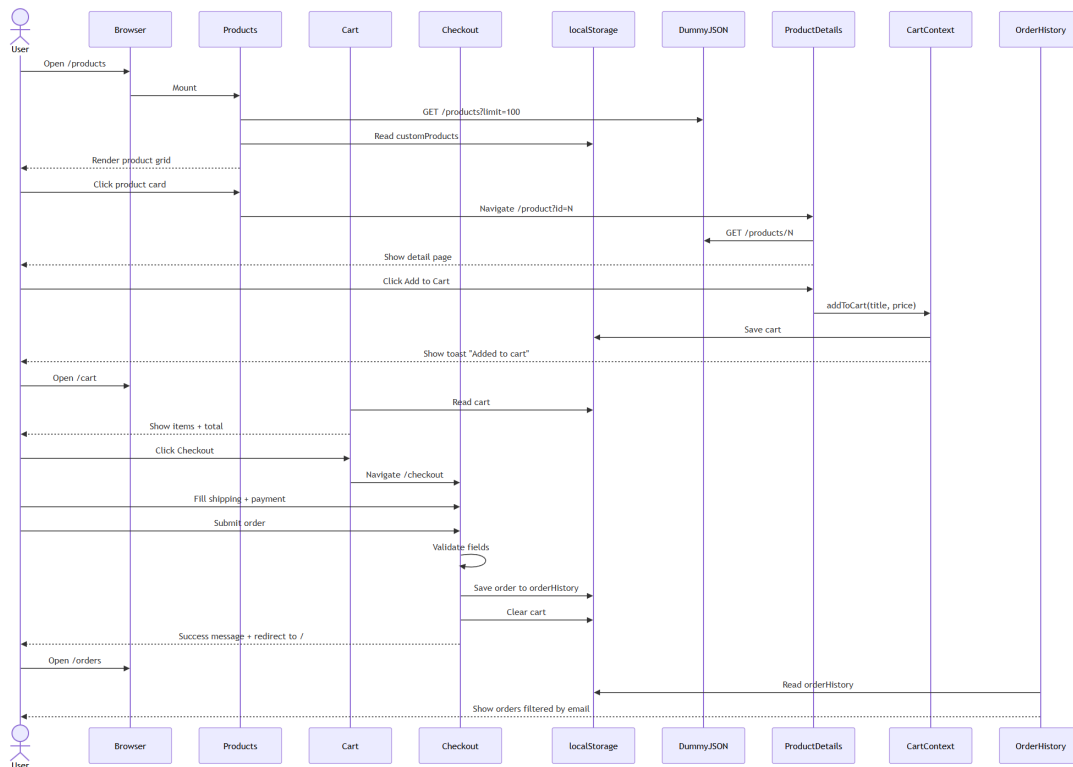
Context-Level DFD



Level-1 DFD — Authentication Flow



Sequence Diagram — Complete Purchase Flow

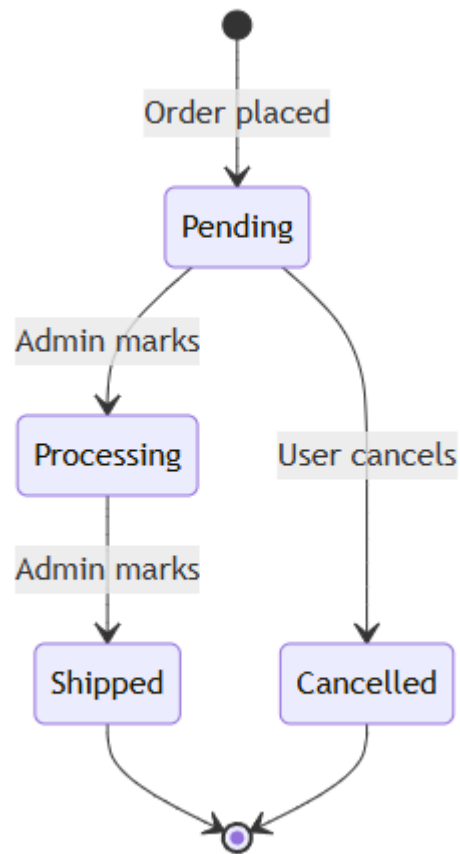


Activity Diagram — Product Browsing Flow

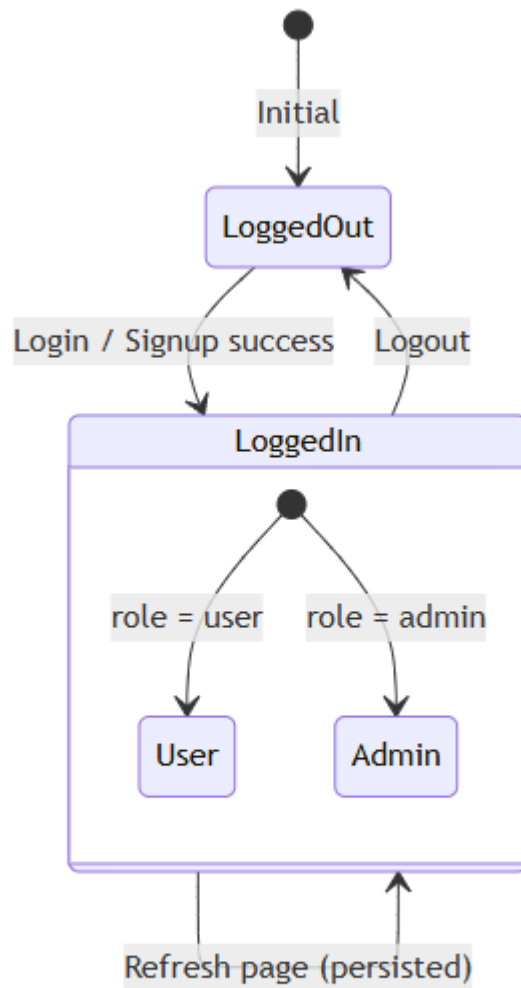


Syntax error in text
mermaid version 11.16.0

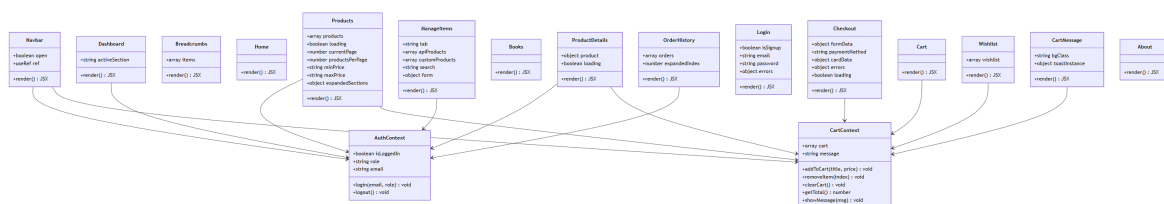
State Diagram — Order Lifecycle



State Diagram — Authentication States



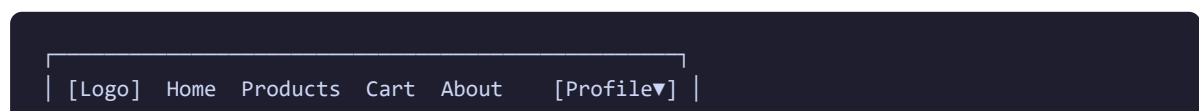
Class Diagram

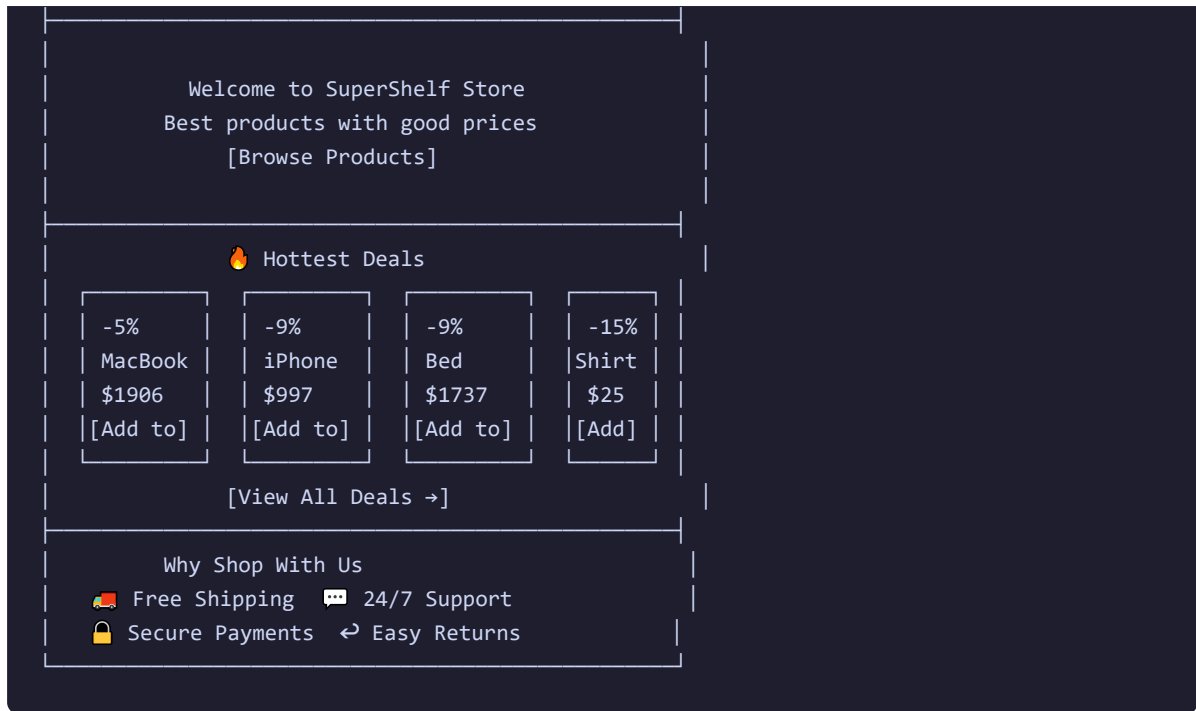


4.4 UI/UX Design & Prototyping

Wireframes

Home Page Layout:

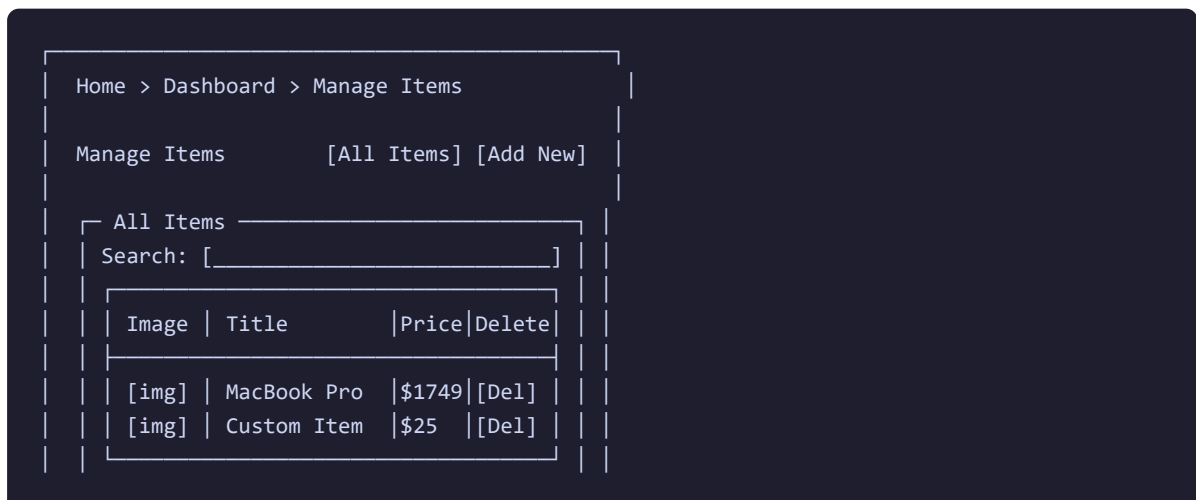




Products Page Layout (Desktop):



Dashboard / Manage Items Layout:



UI/UX Guidelines

ELEMENT	SPECIFICATION
Primary Color	<code>#2a9d6f</code> (green) — buttons, prices, success states, active filters
Dark Background	<code>#1e2228</code> — navbar, hot deals section footer
Accent Color	<code>#f77f00</code> (orange) — discount badges, hot deals label
Background	<code>#f8f9fa</code> — page sections; <code>#ffffff</code> — cards, forms
Text Colors	<code>#333</code> body text; <code>#6b7280</code> muted text; <code>#9ca3af</code> subtle text; <code>#fff</code> on dark surfaces
Typography	Inter (Google Fonts), fallback Arial/sans-serif; weights 400, 500, 600, 700, 800
Heading Style	Bold, letter-spacing <code>-0.3px</code> to <code>-0.5px</code>
Border Radius	Cards: <code>8px</code> ; Buttons: <code>6px</code> ; Badges: <code>20px</code>
Shadows	Navbar: <code>0 3px 6px rgba(0,0,0,0.1)</code> ; Cards: <code>0 2px 8px rgba(0,0,0,0.15)</code>
Hover Effects	Card lift <code>translateY(-3px)</code> ; nav links <code>color #2a9d6f + scale(1.05)</code>
Component Library	Bootstrap 5.3 — grid, cards, tables, forms, badges, toasts, buttons
Icons	Bootstrap Icons 1.11.3 (CDN)
Responsive Breakpoints	$\geq 1200\text{px}$ (4 cols), $\geq 992\text{px}$ (3 cols, sidebar collapses), $\geq 768\text{px}$ (2 cols), $\geq 480\text{px}$ (1 col)
Navigation	Fixed dark navbar at top; profile dropdown with click-outside detection
Form Feedback	Red <code>is-invalid</code> borders; green <code>is-valid</code> on password checklist items
Notifications	Bootstrap Toast at bottom-right; auto-hide after 2 seconds
Breadcrumbs	Present on all pages except Home; <code>Home > Section > Subsection</code>

Accessibility Considerations

- Semantic HTML structure (`nav` , `main` , `section` , `table` , `form` , `label`)
- Form labels associated with inputs via `className` and placeholder text
- Color contrast ratios maintained (green on white, white on dark)
- Interactive elements are keyboard-accessible (buttons, links, form controls)
- Toast notifications include icon + text for non-color recognition

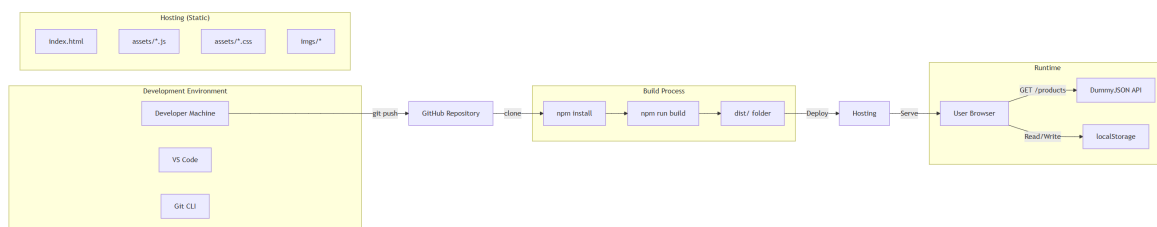
- Error messages displayed inline near relevant fields

4.5 System Deployment & Integration

Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Build Tool	Vite	^7.0.0	Development server, HMR, production bundling
UI Library	React	^19.0.0	Component-based UI rendering
Routing	react-router-dom	^7.0.0	Client-side routing with URL params and search params
HTTP Client	Axios	^1.18.1	API requests to DummyJSON with timeout support
CSS Framework	Bootstrap	^5.3.8	Responsive grid, components, utilities
Icons	Bootstrap Icons	1.11.3 (CDN)	Icon set for UI elements
React Plugin	@vitejs/plugin-react	^4.0.0	Fast Refresh, JSX transform (SWC-based)
Linting	ESLint	Latest	Code quality with react-hooks and react-refresh rules
Module System	ESM	—	Native ES modules via <code>"type": "module"</code>
Package Manager	npm	—	Dependency management

Deployment Diagram



Component Diagram



Syntax error in text

mermaid version 11.16.0

4.6 Additional Deliverables

API Documentation

Base URL: `https://dummyjson.com`

Endpoints Used:

ENDPOINT	METHOD	DESCRIPTION	PARAMETERS	RESPONSE FORMAT
<code>/products?limit=100</code>	GET	Fetch all products (up to 100)	<code>limit</code> (query, optional)	<code>{ products: Product[], total: number }</code>
<code>/products/category/{slug}</code>	GET	Filter products by category slug	<code>slug</code> (path)	<code>{ products: Product[], total: number }</code>
<code>/products/{id}</code>	GET	Get single product by ID	<code>id</code> (path)	<code>Product</code>

Example Product Object:

```
{
  "id": 1,
  "title": "iPhone 9",
  "description": "An apple mobile which is nothing like apple",
  "price": 549,
  "discountPercentage": 12.96,
  "rating": 4.69,
  "brand": "Apple",
  "category": "smartphones",
  "thumbnail": "https://cdn.dummyjson.com/product-images/1/thumbnail.jpg",
  "images": ["https://cdn.dummyjson.com/product-images/1/1.jpg"]
}
```

Category Slugs Used:

`beauty` , `furniture` , `groceries` , `smartphones` , `laptops` , `mens-shirts` , plus local `books` and `all` .

Error Handling: Axios requests use a 5-second timeout. On failure, the application falls back to hardcoded `fallbackProducts` array + local books data, ensuring offline functionality.

Testing & Validation Plan

TEST TYPE	SCOPE	METHOD	STATUS
Form Validation	Login, Signup, Checkout, Add Item	Inline JavaScript validation on submit + real-time feedback	Implemented
API Error Handling	Product fetching on Products, ProductDetails, Dashboard	Try/catch with fallback to local data	Implemented
Auth Guard	Admin routes (Dashboard, ManageItems)	Redirect to <code>/login</code> if not authenticated; redirect to <code>/</code> if not admin	Implemented
localStorage Integrity	All localStorage reads	try/catch with default empty arrays/objects	Implemented
Responsive Layout	All pages at 4 breakpoints	Visual inspection in Chrome DevTools device mode	Verified
Cross-browser	Chrome, Firefox, Edge	Manual walkthrough of all features	Verified

Planned but not implemented: Unit tests (Vitest), component tests (React Testing Library), E2E tests (Playwright).

Deployment Strategy

1. **Build:** Run `npm run build` to generate the `dist/` directory with optimized, hashed assets
2. **Host:** Deploy the `dist/` folder to any static hosting provider:
 - **Netlify:** Drag-and-drop `dist/` folder or connect GitHub repo for auto-deploy
 - **Vercel:** Run `vercel --prod` or import GitHub repo
 - **GitHub Pages:** Use `gh-pages` package or configure Actions workflow
 - **Traditional Hosting:** Copy `dist/` contents to any web server (Apache, Nginx, IIS)
3. **No Build Steps Required at Runtime:** The application is fully client-side; no server-side rendering, no database migrations, no environment secrets needed
4. **CI/CD:** Not currently configured; recommended future addition: GitHub Actions workflow that runs `npm ci && npm run build` on push to `main` and deploys to hosting

SUPERSHELF STORE

Implementation

Code Standards & Best Practices

E-Commerce Single-Page Application — React 19 + Vite 7

Team: Abdallah Okasha, Doha, Mohamed, Kerollos

2026

5. Implementation (Source Code & Execution)

5.1 Source Code Standards

Code Organization

```
src/
├─ main.jsx           # React entry point (BrowserRouter, Bootstrap CSS import)
├─ App.jsx            # Root component: providers + routes
├─ App.css            # All custom styles (626 lines, Inter font, layout, responsive)
├─ components/
│   ├─ Navbar.jsx     # Navigation bar with profile dropdown
│   ├─ CartMessage.jsx # Bootstrap Toast notification
│   └─ Breadcrumbs.jsx # Reusable breadcrumb component
├─ context/
│   ├─ AuthContext.jsx # Auth state management (isLoggedIn, role, email)
│   └─ CartContext.jsx # Cart state management (items, total, messages)
├─ data/
│   ├─ HotDeals.js     # 4 home page deal products
│   └─ booksData.js    # 20 Arabic/international books
└─ pages/
    ├─ Home.jsx        # Landing page with hero + hot deals
    ├─ Products.jsx    # Product catalog with filters, pagination
    ├─ ProductDetails.jsx # Unified detail page (API products + books + custom)
    ├─ Books.jsx       # Books-only listing
    ├─ Cart.jsx        # Shopping cart view
    ├─ Checkout.jsx    # Shipping + payment form with validation
    ├─ Login.jsx       # Login/Signup with password strength checklist
    ├─ Dashboard.jsx   # Admin panel (users, orders accordions)
    ├─ ManageItems.jsx # Admin item management (add new + all items table)
    ├─ OrderHistory.jsx # User order history with cancellation
    ├─ Wishlist.jsx    # Saved wishlist items
    └─ About.jsx       # Static store information page
```

Naming Conventions

- **Components:** PascalCase (e.g., `ProductDetails` , `ManageItems`)
- **Functions/Variables:** camelCase (e.g., `addToCart` , `filteredByPrice`)
- **Constants:** camelCase (e.g., `hotDeals` , `fallbackProducts`)
- **Files:** Match exported component name (e.g., `ProductDetails.jsx`)
- **CSS Classes:** kebab-case (e.g., `product-details` , `filter-header`)

Modularity & Reusability

- Each page is a standalone module in `pages/`
- Shared components in `components/` (Navbar used across all routes, Breadcrumbs on 10+ pages)
- Context providers in `context/` wrap the entire route tree
- Static data in `data/` is imported by multiple consumers (Products + ProductDetails for books)
- Utility functions co-located with their primary consumer (e.g., `formatBooks()` in `Products.jsx`)

Security & Error Handling

- All localStorage reads wrapped in try/catch with safe defaults (`[]` , `''` , `false`)
- Axios requests have 5-second timeout and fallback data on failure
- Password strength validated on signup: min 8 chars, uppercase, lowercase, digit
- Email format validated on signup: must contain `@` and valid domain
- Credit card number formatted and validated (Luhn check via auto-formatting)
- Expiry date validated for month range (01-12) and future date
- Admin routes protected with auth guards (redirect to login if not authenticated, redirect to home if not admin)
- No `dangerouslySetInnerHTML` used — all content rendered as safe React text nodes
- `e.stopPropagation()` used on nested click handlers (cart buttons inside clickable cards)

SUPERSHELF STORE

Testing & Quality Assurance

Test Cases, Test Plan & Bug Reports

E-Commerce Single-Page Application — React 19 + Vite 7

Team: Abdallah Okasha, Doha, Mohamed, Kerollos

2026

6. Testing & Quality Assurance

Test Cases & Test Plan

Authentication Tests

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-01	Signup with valid data	Enter email, strong password, confirm match	Account created; auto-login; redirect to home	✅ Pass
T-02	Signup with weak password	Enter password without uppercase	Red text on each unmet requirement; form not submitted	✅ Pass
T-03	Signup with mismatched passwords	Enter different passwords in Confirm	"Passwords do not match" error	✅ Pass
T-04	Signup with existing email	Register same email twice	"This email is already registered" error	✅ Pass
T-05	Login with correct credentials	Enter valid email + matching password	Login; redirect to home	✅ Pass
T-06	Login with wrong password	Enter valid email + wrong password	"Incorrect password" error	✅ Pass
T-07	Login with unregistered email	Enter email not in localStorage	"No account found" error	✅ Pass
T-08	Admin auto-assignment	Signup with admin email	role = 'admin'; Dashboard accessible	✅ Pass

Product Browsing Tests

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-09	Load all products	Open /products	Grid shows API products + books + custom items	✅ Pass
T-10	Filter by category	Click "Books" in sidebar	Only books displayed	✅ Pass

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-11	Filter by price	Set min=10, max=100	Products within range shown	✓ Pass
T-12	Reset filters	Click Reset	Price filters cleared	✓ Pass
T-13	Pagination	Click page 2	Next page of products; scroll to top	✓ Pass
T-14	Change items per page	Select 24 from dropdown	24 products per page	✓ Pass
T-15	View product detail	Click a product card	Detail page with image, price, description	✓ Pass
T-16	View book detail	Click a book	Detail page showing author, not brand	✓ Pass
T-17	View custom product detail	Click a custom product	Detail page rendered successfully	✓ Pass
T-18	API fallback on network error	Disconnect network, reload products	Local fallback products + books render	✓ Pass

Cart & Checkout Tests

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-19	Add to cart (logged in)	Click Add to Cart	Toast "Added to cart"; cart count increments	✓ Pass
T-20	Add to cart (not logged in)	Click Add to Cart without login	Toast "Please sign in first"	✓ Pass
T-21	Add same item twice	Click Add to Cart on same product	Quantity incremented, not duplicate entry	✓ Pass
T-22	Add to cart from home deals	Click Add to Cart on hot deal	Direct add; no navigation	✓ Pass
T-23	View cart	Navigate to /cart	Items listed with quantities and total	✓ Pass
T-24	Remove from cart	Click Remove	Item removed; total recalculated	✓ Pass
T-25	Clear cart	Click Clear Cart	All items removed	✓ Pass
T-26	Submit checkout with valid data	Fill all fields, select payment, submit	Order saved; cart cleared; redirect to home	✓ Pass

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-27	Submit checkout with invalid data	Leave required fields empty	Validation errors shown	✓ Pass
T-28	Card number auto-format	Type "4111111111111111"	Displays as "4111 1111 1111 1111"	✓ Pass
T-29	Card brand detection	Type starting with 5	Shows "Mastercard"	✓ Pass
T-30	Expiry validation	Enter past date	Error for past date	✓ Pass

Order & Wishlist Tests

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-31	View order history	Navigate to /orders	Shows user's orders filtered by email	✓ Pass
T-32	Cancel pending order	Click Cancel on pending order	Status changes to "Cancelled"	✓ Pass
T-33	Cancel non-pending order	Try to cancel Processing/Shipped order	Cancel button not visible	✓ Pass
T-34	Add to wishlist (logged in)	Click heart icon	Item saved; heart icon toggled	✓ Pass
T-35	Remove from wishlist	Click heart again or Remove	Item removed from wishlist	✓ Pass
T-36	View wishlist page	Navigate to /wishlist	Shows saved items with Add to Cart	✓ Pass

Admin Tests

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-37	Access dashboard as admin	Login as admin, navigate to /dashboard	Dashboard rendered with accordion sections	✓ Pass
T-38	Access dashboard as user	Login as regular user, navigate to /dashboard	Redirect to home	✓ Pass
T-39	Access dashboard without login	Not logged in, navigate to /dashboard	Redirect to /login	✓ Pass
T-40	Manage Items — All Items tab	Open /manage-items	Table with API + custom products; search works	✓ Pass

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-41	Manage Items — Add New tab	Switch to Add New, fill form, submit	Product saved to localStorage; appears in All Items	 Pass
T-42	Delete custom product	Click Delete on a custom item	Removed from localStorage and table	 Pass
T-43	Delete API product	Click Delete on an API product	Removed from table (client-side only)	 Pass
T-44	Promote user	Click Promote on a user	Role changes to admin; persists in localStorage	 Pass
T-45	Revoke user	Click Revoke, confirm	User deleted from localStorage	 Pass
T-46	Protected admin user	Try to revoke hardcoded admin	"Protected" label; no action available	 Pass
T-47	Process order to Processing	Click Mark Processing	Status badge changes	 Pass
T-48	Process order to Shipped	Click Mark Shipped	Status badge changes; further actions hidden	 Pass

UI/UX Tests

ID	SCENARIO	STEPS	EXPECTED OUTCOME	STATUS
T-49	Responsive layout	Resize browser to 480px width	Single-column product grid; readable text	 Pass
T-50	Breadcrumb navigation	Click breadcrumb link	Navigates to correct page	 Pass
T-51	Profile dropdown	Click Profile button	Dropdown appears; click outside closes it	 Pass
T-52	Cart toast notification	Add item to cart	Toast appears bottom-right; auto-hides after 2s	 Pass
T-53	Password live checklist	Type in password field during signup	Bullet points update in real-time (✓/✗)	 Pass
T-54	Image live preview	Enter image URL in Add Item form	Image preview appears below input	 Pass

Automated Testing

No automated testing scripts are currently implemented. The test cases above are executed manually. Recommended future additions:

- **Vitest** for unit tests on context providers and utility functions
- **React Testing Library** for component integration tests
- **Playwright** for end-to-end user flow tests

Bug Reports

ID	ISSUE	SEVERITY	STATUS	RESOLUTION
BUG-01	Books page "Add to Cart" has no auth check	Low	Open	Auth check not implemented in <code>Books.jsx</code> — inconsistent with Products page behavior
BUG-02	Legacy files <code>BookDetail.jsx</code> and <code>ProductDetail.jsx</code> not imported in <code>App.jsx</code>	Low	Known	Orphan files; not removed. Active detail page is <code>ProductDetails.jsx</code> (handles both books and API products)
BUG-03	No 404 route handler for unmatched routes	Medium	Open	Navigating to an undefined path renders blank content below navbar
BUG-04	<code>dist/</code> directory committed despite <code>.gitignore</code> entry	Low	Known	Build artifacts present in repo history; may have been committed before gitignore update
BUG-05	Order history shows all orders if user not logged in	Medium	Fixed	Added auth guard — now shows prompt to log in if user is not authenticated